CS-315 | Parallel and Distributed Computing

Parallel DNA Pattern Matching for Neurological Disease Detection

Submitted To: Dr. Muhammad Tariq Saeed

Submitted By:

Arooj Fatima	REG-No: 475958
Maryam Sajid	REG-No: 459508
Muhammad Abdullah Mustafa	REG-No: 458817
Saleha Asim	REG-No: 459307

Batch: BS Bioinformatics '23 UG - 1 | SINES

National University of Sciences and Technology (NUST)

April 9, 2026

Problem Statement

The human genome is a string of roughly 3 billion nucleotide characters from the alphabet {A, T, G, C}. A family of neurological disorders, including Huntington's disease (CAG repeats in HTT), Spinocerebellar Ataxia subtypes (SCA1, SCA2, SCA3), Fragile X syndrome (CGG in FMR1), and Friedreich's Ataxia (GAA in FXN), are all caused by the abnormal expansion of short repeating DNA sequences called trinucleotide repeats. Searching chromosome 1 alone (249 million characters) for these patterns takes $\sim$10 seconds sequentially. Scaled to a full genome or a patient batch, this becomes a genuine bottleneck that MPI parallelism can directly address.

Dataset

Chromosome 1 from the GRCh38 human reference genome (UCSC) was used as input, producing a 238 MB raw string of 248,956,422 characters after stripping FASTA headers. Since the reference genome is healthy and returns zero matches, 8 synthetic mutation sites were injected at known positions using a C program, each consisting of 40–70 pathological repeats of CAG, CGG, CTG, GAA, and CAA. The resulting diseased genome yields **791 matches** across 12 disease marker patterns. The match count reflects overlapping hits across patterns of different lengths targeting the same injected regions.

Decomposition Schemes

Three parallel strategies were implemented, each decomposing the problem differently. Figure 1 illustrates the distribution of work across MPI processes for all three schemes.

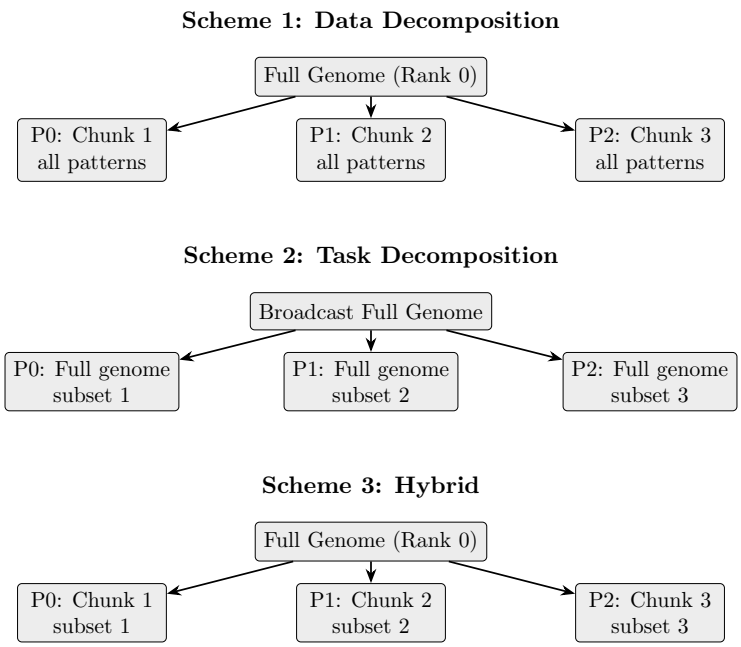

Figure 1: The three MPI decomposition strategies.

Scheme 1 — Data Decomposition

The genome is split into equal chunks, one per process. Rank 0 reads and distributes chunks via point-to-point sends. Each chunk includes an overlap of $m - 1$ characters at its boundary (where m is the longest pattern length) to avoid missing cross-boundary matches. Every process searches all 12 patterns in its chunk using naive string matching and reports matches via `MPI_Reduce`.

Processes	Time (s)	Speedup
1 (sequential)	9.93	1.00×
2	5.44	1.83×
4	3.05	3.26×
8	2.03	4.89×
16	1.20	8.28×

Table 1: Scheme 1 results. Speedup reaches 8.28× at 16 processes. The gap from ideal linear speedup is due to data scatter overhead and boundary overlap cost.

Scheme 2 — Task Decomposition

The full genome is broadcast to all processes via `MPI_Bcast`. Patterns are divided evenly across processes using a balanced allocation, so each process receives either $\lfloor pat_count/size \rfloor$ or $\lfloor pat_count/size \rfloor + 1$ patterns. Each process scans the entire genome for only its assigned patterns using naive matching, and local match counts are aggregated via `MPI_Reduce`. No boundary overlap is required since genome partitioning is not involved. This scheme trades memory for simplicity: every process holds a full copy of the genome, which increases memory usage but eliminates scatter communication.

Processes	Time (s)	Speedup
1 (sequential)	9.93	1.00×
2	7.04	2.01×
4	3.84	3.66×
8	2.55	5.51×
16	1.75	8.03×

Table 2: Scheme 2 results with balanced pattern distribution. Speedup improves consistently across all process counts. Performance is bounded by the number of patterns available for distribution.

Scheme 3 — Hybrid Decomposition

Hybrid decomposition partitions both the genome and the patterns simultaneously. Each process receives a chunk of the genome and searches all 12 patterns within its chunk, with a boundary overlap of 255 characters to handle cross-boundary matches. This combines the data reduction benefit of Scheme 1 with the communication simplicity of Scheme 2, and achieves the best scaling across all process counts.

Processes	Time (s)	Speedup
1 (sequential)	9.93	1.00×
2	6.2268	1.59×
4	2.7246	3.64×
8	1.7915	5.54×
16	0.9024	11.00×

Table 3: Scheme 3 results. Highest scalability across all three schemes, reaching 11.00× speedup at 16 processes.

Performance Comparison

Figure 2 shows the speedup curves for all three schemes across 1, 2, 4, 8, and 16 MPI processes, plotted against the ideal linear speedup baseline.

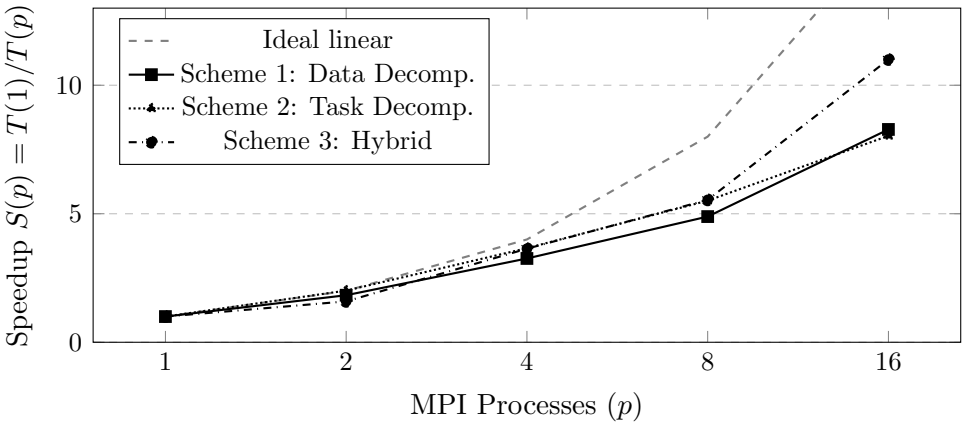

Figure 2: Speedup $S(p) = T(1)/T(p)$ for all three decomposition schemes. Wall time measured via `MPI_Wtime()`. Genome: 248,956,422 characters (238 MB).

All three schemes demonstrate consistent speedup with increasing process counts. Scheme 3 achieves the highest scalability at 11.00× with 16 processes, followed by Scheme 1 at 8.28× and Scheme 2 at 8.03×.

Scheme 1 scales well because it reduces the per-process search space directly proportional to process count, with only minor overhead from data scatter and boundary overlap. Scheme 2 scales effectively when patterns are distributed in a balanced manner, though it is ultimately bounded by the number of available patterns and the cost of holding the full genome in memory on every process.

Scheme 3 benefits from both data reduction and a large overlap buffer, which together allow it to outperform the other two at higher process counts. While it incurs a slight overhead at 2 processes compared to Scheme 1, it provides the best execution time at 16 processes (0.90s). Consequently, for large-scale genomic pattern matching where both the dataset size and the pattern library are significant, a hybrid approach is the optimal choice for high-performance computing environments.

References

[1] Gonzalez-Escribano, A. et al. (2024). DNA sequence alignment: An assignment for OpenMP, MPI, and CUDA/OpenCL. *arXiv:2409.06075*.

[2] Lobeiras, J. et al. (2021). MPI-dot2dot: A parallel tool to find DNA tandem repeats on multicore clusters. *Journal of Supercomputing*, 78, 4166–4188.

[3] Mistry, J. & Bhatt, D. (2024). Trinucleotide Repeat Disorders. *StatPearls*, NCBI Bookshelf NBK559254.